

计算机设计实践

最终实验报告

单周期 CPU 与五级流水线 CPU 扩展指令验收

姓名	黄宇轩
学号	2024302181173
实验内容	RISC-V 单周期 CPU 与流水线 CPU 指令扩展、仿真验收与上板展示
报告日期	2026 年 6 月 17 日
GitHub	https://github.com/yuxuanhuang641-crypto/computer-design-practice-final-2024302181173

本报告依据 final/README.md、QUESTIONS.md、老师补充要求、代码 diff、四组验收测试截图和开发板照片整理。报告正文不展开完整源码，只保留关键实现片段、原理说明和结果证据。

一、实验任务与完成情况

最终验收要求在原有 RISC-V 单周期 CPU 和五级流水线 CPU demo 基础上补充扩展指令，并能够通过 iverilog 测试、学号排序程序和开发板展示。本次修改围绕译码、ALU 运算、立即数扩展、下一条 PC 计算、流水线转发和控制冲刷展开。

CPU 类型	要求实现的新增/重点指令	完成说明
单周期 CPU	slt/sltu、andi/ori/xori、srli/srai/slli、slli/sltiu、bne/bge/bgeu/blt/bltu、jalr	通过 ctrl 译码、EXT 立即数扩展、ALU 运算/比较、NPC 跳转目标计算完成。
流水线 CPU	单周期范围基础上增加 beq、jal、jalr 的流水线支持	在五级流水寄存器中传递控制信号，增加转发路径和跳转 flush，保证后续指令不使用错误数据或错误 PC。

自动评分脚本结果如下，所有小测试点均通过，得分 100/100。

图 1 自动测试结果：Score 100/100

```
PS E:\computerlab\final_acceptance> python tools\run_score_all.py --no-compile
[PASS] sc_slt_signed: 4/4
[PASS] sc_sltu_unsigned: 4/4
[PASS] sc_logic_immediate: 6/6
[PASS] sc_shift_immediate: 5/5
[PASS] sc_slti_sltiu: 7/7
[PASS] sc_branch_group: 10/10
[PASS] sc_jalr_link_jump: 6/6
[PASS] pl_slt_signed: 4/4
[PASS] pl_sltu_unsigned: 4/4
[PASS] pl_logic_immediate: 6/6
[PASS] pl_shift_immediate: 5/5
[PASS] pl_slti_sltiu: 7/7
[PASS] pl_branch_group: 10/10
[PASS] pl_beq_flush: 2/2
[PASS] pl_jal_flush: 6/6
[PASS] pl_jalr_flush: 6/6
Score: 100/100
Raw score: 92/92
```

二、总体实现原理

2.1 单周期 CPU 数据通路

单周期 CPU 的核心特点是一条指令在一个时钟周期内走完完整条数据通路：PC 取指，ctrl 根据 opcode/funct3/funct7 产生控制信号，RF 读出 rs1/rs2，EXT 产生立即数，ALU 完成运算或分支比较，dm 在 lw/sw 时访存，最后由 WDSel 选择 ALU、内存或 PC+4 写回寄存器。

单周期修改重点：补充指令译码、扩充 ALUOp、增加 jalr 的 RS1 输入和跳转目标计算。因为单周期没有阶段重叠，所以不需要转发和阻塞，控制信号在当前周期内直接使用。

2.2 五级流水线 CPU 数据通路

流水线 CPU 将一条指令拆成 IF、ID、EX、MEM、WB 五个阶段。每个阶段只做一部分工作，阶段之间用 IF/ID、ID/EX、EX/MEM、MEM/WB 流水寄存器保存数据和控制信号。这样多个指令可以重叠执行，但也会出现数据冒险和控制冒险。

流水寄存器	保存内容	作用
IF/ID	PC、当前指令 instr	把取指结果送到译码阶段。
ID/EX	rd/rs1/rs2、RD1/RD2、imm、ALUOp、NPCOp、RegWrite、MemRead/MemWrite、WDSel	让译码结果跟着同一条指令进入执行阶段。
EX/MEM	PC、rd、ALU 结果、写内存数据、访存控制信号、写回选择	把执行结果送到访存阶段，同时保留后续写回信息。
MEM/WB	PC、rd、ALU 结果、内存读数、RegWrite、WDSel	在 WB 阶段选择最终写回值并写入 RF。

三、老师可能提问的指令实现说明

3.1 运算指令示例：add / slt

`add` 是最典型的 R-type 运算指令，`slt` 是本次扩展中容易被问到的比较运算指令。二者共同点是都从 RF 读两个源寄存器，ALUSrc=0，结果来自 ALU，RegWrite=1 写回 rd。区别是 `add` 输出 A+B，而 `slt` 输出比较结果 0 或 1。

- 1. 取指：im 根据 PC[31:2] 取出 32 位机器码。
- 2. 译码：ctrl 读取 Op=0110011，判断为 R-type，再用 {Funct7, Funct3} 区分 add、sub、slt、sltu 等。
- 3. 读寄存器：RF 根据 rs1、rs2 输出 RD1、RD2。
- 4. 执行：ALU 按 ALUOp 执行加法或比较。slt 成立输出 1，否则输出 0。
- 5. 写回：WDSel 选择 ALU 输出，RegWrite=1 时写入 rd；x0 例外，不会被写坏。

代码片段 1 运算指令在 ctrl 中被识别为具体 ALUOp

```
// single-cycle/ctrl.v: R-type 译码
7'b0110011: begin
    RegWrite = 1'b1;
    case ({Funct7, Funct3})
        {7'b0000000, 3'b000}: ALUOp = `ALUOp_add;
        {7'b0100000, 3'b000}: ALUOp = `ALUOp_sub;
        {7'b0000000, 3'b010}: ALUOp = `ALUOp_slt;
        {7'b0000000, 3'b011}: ALUOp = `ALUOp_sltu;
    endcase
end
```

代码片段 2 add/slt/sltu 的实际运算位置

```
// single-cycle/alu.v: ALU 执行
`ALUOp_add: C = A + B;
`ALUOp_sub: C = A - B;
`ALUOp_slt: C = (A < B) ? 32'd1 : 32'd0;
`ALUOp_sltu: C = ($unsigned(A) < $unsigned(B)) ? 32'd1 : 32'd0;
```

3.2 跳转/分支指令示例: bne / jalr

`bne` 是条件分支，判断 RD1 与 RD2 是否不相等；如果条件成立，NPC 选择 PC+branch_imm，否则继续 PC+4。`jalr` 是寄存器间接跳转，目标地址来自 RS1+imm，并按 RISC-V 要求把最低位清 0。

6. bne 译码时 Op=1100011、Funct3=001，ctrl 设置 EXTOp=BTYPE，ALUOp=bne。
7. ALU 判断 A!=B，把成立与否编码到 C；Zero 信号在分支类 ALUOp 下表示“条件成立”。
8. 如果 Zero 为真，ctrl 输出 NPC_BRANCH，NPC 用 PC+IMM 得到分支目标；否则 NPC_PLUS4 顺序执行。
9. jalr 译码时 Op=1100111，ctrl 让 RegWrite=1、WDSel=PC+4、NPCOp=NPC_JALR；NPC 用 RS1+IMM 计算跳转目标。

代码片段 3 bne 条件成立才分支, jalr 写回 PC+4 并跳转

```
// single-cycle/ctrl.v: bne 与 jalr 控制
7'b1100011: begin
    EXTOp = `EXT_CTRL_BTTYPE;
    case (Funct3)
        3'b001: begin ALUOp = `ALUOp_bne;
                    NPCOp = Zero ? `NPC_BRANCH : `NPC_PLUS4; end
    endcase
end

7'b1100111: begin
    RegWrite = 1'b1;
    ALUSrc = 1'b1;
    EXTOp = `EXT_CTRL_ITYPE;
    NPCOp = `NPC_JALR;
    WDSel = `WDSel_FromPC;
end
```

代码片段 4 jalr 的目标地址需要 RS1, 因此 NPC 模块新增 RS1 输入

```
// single-cycle/NPC.v: 下一条 PC 计算
`NPC_PLUS4: NPC = PCPLUS4;
`NPC_BRANCH: NPC = PC + IMM;
`NPC_JUMP: NPC = PC + IMM;
`NPC_JALR: NPC = (RS1 + IMM) & 32'hffff_fffe;
```

四、单周期与流水线的区别，以及冒险处理

4.1 单周期和流水线实现差异

对比项	单周期 CPU	流水线 CPU
执行方式	一条指令一个周期内完成全部步骤。	一条指令分 IF/ID/EX/MEM/WB 多个阶段，多条指令重叠执行。
控制信号	译码后本周期直接使用。	控制信号必须随指令进入 ID/EX、EX/MEM、MEM/WB。
PC 更新	分支/跳转在当前组合逻辑中确定下一 PC。	分支/跳转通常到 EX 阶段才确定，需要 flush 前面取错的指令。
数据冒险	没有阶段重叠，基本不出现 RAW 冒险。	后续指令可能提前需要前一条指令尚未写回的值，需要 forwarding 或 stall。

4.2 RAW 数据冒险与转发

RAW 冒险就是 Read After Write：后一条指令要读的寄存器，正好是前一条或前几条指令还没有写回的 rd。例如 `add x3,x1,x2` 后面紧跟 `sub x4,x3,x5`，sub 在 EX 阶段已经需要 x3，但 add 的结果还在 EX/MEM 或 MEM/WB 中。

本工程在 PLCPU 中增加 `mem\_forward\_value`、`wb\_forward\_value`、`forward\_RD1`、`forward\_RD2`。如果 EX 阶段源寄存器等于 MEM 或 WB 阶段将要写回的 rd，就不等 RF 写回，而是直接把结果送回 ALU 输入端。

代码片段 5 流水线数据转发逻辑

```
assign mem_forward_value =
    (MEM_WDSEL == `WDSEL_FromPC) ? (EX_MEM_out[31:0] + 4) :
    (MEM_WDSEL == `WDSEL_FromMEM) ? Data_in :
    MEM_aluout;

assign forward_RD1 =
    (MEM_RegWrite && (MEM_rd != 5'b0) && (MEM_rd == EX_rs1)) ? mem_forward_value :
    (WB_RegWrite && (WB_rd != 5'b0) && (WB_rd == EX_rs1)) ? wb_forward_value :
    EX_RD1;

assign forward_RD2 =
    (MEM_RegWrite && (MEM_rd != 5'b0) && (MEM_rd == EX_rs2)) ? mem_forward_value :
    (WB_RegWrite && (WB_rd != 5'b0) && (WB_rd == EX_rs2)) ? wb_forward_value :
    EX_RD2;
```

关于 load-use：经典 load-use 冒险是 `lw x1,0(x2)` 后面立即使用 `x1`。此时数据要到 MEM 阶段才从 dm 出来，最稳妥的做法是检测到 `EX\_MemRead && EX\_rd == ID\_rs` 后冻结 PC/IF_ID 一拍并向 ID/EX 插入 bubble。本工程当前验收程序主要通过 MEM/WB 转发路径覆盖数据相关，报告中需要说明 load-use 的原理和标准解决方法，避免把普通转发误说成所有 load-use 都已经零等待解决。

4.3 控制冒险与 flush

分支和 jal/jalr 改变 PC。流水线在跳转真正确定前，IF/ID 和 ID/EX 可能已经进入了错误路径指令，因此本工程用 `jump\_flush` 清空这些寄存器输入，相当于插入 NOP。

代码片段 6 控制冒险处理：跳转/分支成立时冲刷流水线前级

```
wire jump_flush;
assign jump_flush = (EX_NPCOp != `NPC_PLUS4);

assign IF_ID_in = jump_flush ? 64'b0 : IF_ID_raw_in;
assign ID_EX_in = jump_flush ? 194'b0 : ID_EX_raw_in;
```

五、关键模块说明

im 指令存储器：保存机器码。PC 按字节计数，而指令按 32 位 word 存储，因此常用 PC[31:2] 作为 im 地址。testbench 通过 \$readmemh 加载 .dat 程序。

RF 寄存器堆：包含 x0-x31，两个读端口和一个写端口。rs1/rs2 组合读出，rd 在时钟沿写回；x0 永远保持 0。

dm 数据存储器：服务 lw/sw。sw 在 MemWrite=1 时按地址写入 dmem，lw 读出数据并在 WB 阶段写回 rd。工程按 32 位 word 访问，所以地址常用 addr[8:2]。

EXT 立即数扩展：根据 I/S/B/U/J 类型拼接并符号扩展立即数。B/J 型立即数最后补 0，是因为跳转偏移以 2 字节对齐。

ctrl 译码模块：读取 opcode、funct3、funct7，产生 RegWrite、MemWrite、EXTOp、ALUOp、NPCOp、ALUSrc、WDSel 等控制信号。

NPC 下一条 PC 模块：根据 NPCOp 选择 PC+4、PC+branch_imm、PC+jump_imm 或 (RS1+imm)&~1，实现顺序执行、分支、jal、jalr。

六、测试结果与截图

老师要求四个窗口分别展示四组测试的寄存器/内存关键结果。下面先列出关键检查值，再给出对应截图。

测试点	CPU	程序	关键结果
测试 1	单周期	Test_30_Instr.dat	x1=000000cc, x4=00000001, x27=3dcc0000, m0=0000000c, m4=0000055c
测试 2	流水线	Test_30_Instr.dat	寄存器与内存关键值和单周期一致，说明转发/跳转处理满足该程序。
测试 3	单周期	riscv_sidascsorting_sim.dat	x15=03345578, m96=54873530, m97=03345578
测试 4	流水线	riscv_sidascsorting_sim.dat	x15=03345578, 写 memaddr=0x180/0x184, m96=54873530, m97=03345578

图 2 测试 1: 单周期 CPU 跑 Test_30_Instr.dat 的寄存器截图

```
... range parameters $readmemh( ... , mem, $start, $stop); Defaulting to 1504 2000 behav
WARNING: ..\tb\sc_final_tb.v:24: $readmemh(E:\computerlab\ComputerDesignLabs-main\CODExp
127].
[SNAPSHOT] registers
[REG] x0=00000000
[REG] x1=000000cc
[REG] x2=00000000
[REG] x3=00000000
[REG] x4=00000001
[REG] x5=98763dcc
[REG] x6=98765000
[REG] x7=00001236
[REG] x8=98764c00
[REG] x9=0000000c
[REG] x10=00000561
[REG] x11=00000000
[REG] x12=00000000
[REG] x13=00000000
[REG] x14=00000000
[REG] x15=00000000
[REG] x16=00000000
[REG] x17=00000000
[REG] x18=00000500
[REG] x19=98763dcc
[REG] x20=00000001
[REG] x21=98765001
[REG] x22=98766236
[REG] x23=00001236
[REG] x24=00000000
[REG] x25=98767236
[REG] x26=00000236
[REG] x27=3dcc0000
[REG] x28=098763dc
[REG] x29=f98763dc
[REG] x30=00000000
[REG] x31=00000000
```


图 3 测试 1: 单周期 CPU 跑 Test_30_Instr.dat 的内存截图

```
[SNAPSHOT] memory
[MEM] m0=0000000c
[MEM] m1=98765001
[MEM] m2=00001236
[MEM] m3=00000561
[MEM] m4=0000055c
[MEM] m5=00000000
[MEM] m6=00000000
[MEM] m7=00000000
[MEM] m8=00000000
[MEM] m9=00000000
[MEM] m10=00000000
[MEM] m11=00000000
[MEM] m12=00000000
[MEM] m13=00000000
[MEM] m14=00000000
[MEM] m15=00000000
[MEM] m16=00000000
[MEM] m17=00000000
[MEM] m18=00000000
[MEM] m19=00000000
[MEM] m20=00000000
[MEM] m21=00000000
[MEM] m22=00000000
[MEM] m23=00000000
[MEM] m24=00000000
[MEM] m25=00000000
[MEM] m26=00000000
[MEM] m27=00000000
[MEM] m28=00000000
[MEM] m29=00000000
[MEM] m30=00000000
[MEM] m31=00000000
[MEM] m32=00000000
[MEM] m33=00000000
[MEM] m34=00000000
[MEM] m35=00000000
[MEM] m36=00000000
[MEM] m37=00000000
[MEM] m38=00000000
[MEM] m39=00000000
[MEM] m40=00000000
[MEM] m41=00000000
[MEM] m42=00000000
[MEM] m43=00000000
[MEM] m44=00000000
[MEM] m45=00000000
[MEM] m46=00000000
[MEM] m47=00000000
```

图 4 测试 2: 流水线 CPU 跑 Test_30_Instr.dat 的寄存器截图

```
WARNING: ..\tb\pl_final_tb.v:22: $readmemh(E:\computerlab\ComputerDesignLabs-main\CODE
memaddr = 00000000, memdata = 98763dcc
memaddr = 00000004, memdata = 98765001
memaddr = 00000008, memdata = 00001236
memaddr = 0000000c, memdata = 00000000
memaddr = 00000010, memdata = 0000000c
memaddr = 00000014, memdata = 0000055c
memaddr = 00000018, memdata = 00000561
memaddr = 0000001c, memdata = 00000571
[SNAPSHOT] registers
[REG] x0=00000000
[REG] x1=000000cc
[REG] x2=00000000
[REG] x3=00000000
[REG] x4=00000001
[REG] x5=98763dcc
[REG] x6=98765000
[REG] x7=00001236
[REG] x8=98764c00
[REG] x9=0000000c
[REG] x10=00000561
[REG] x11=00000000
[REG] x12=00000000
[REG] x13=00000000
[REG] x14=00000000
[REG] x15=00000000
[REG] x16=00000000
[REG] x17=00000000
[REG] x18=00000500
[REG] x19=98763dcc
[REG] x20=00000001
[REG] x21=98765001
[REG] x22=98766236
[REG] x23=00001236
[REG] x24=00000000
[REG] x25=98767236
[REG] x26=00000236
[REG] x27=3dcc0000
[REG] x28=098763dc
[REG] x29=f98763dc
[REG] x30=00000000
[REG] x31=00000000
[SNAPSHOT] memory
[MEM] m0=0000000c
[MEM] m1=98765001
[MEM] m2=00001236
[MEM] m3=00000561
[MEM] m4=0000055c
[MEM] m5=00000000
[MEM] m6=00000000
[MEM] m7=00000000
[MEM] m8=00000000
[MEM] m9=00000000
[MEM] m10=00000000
[MEM] m11=00000000
```

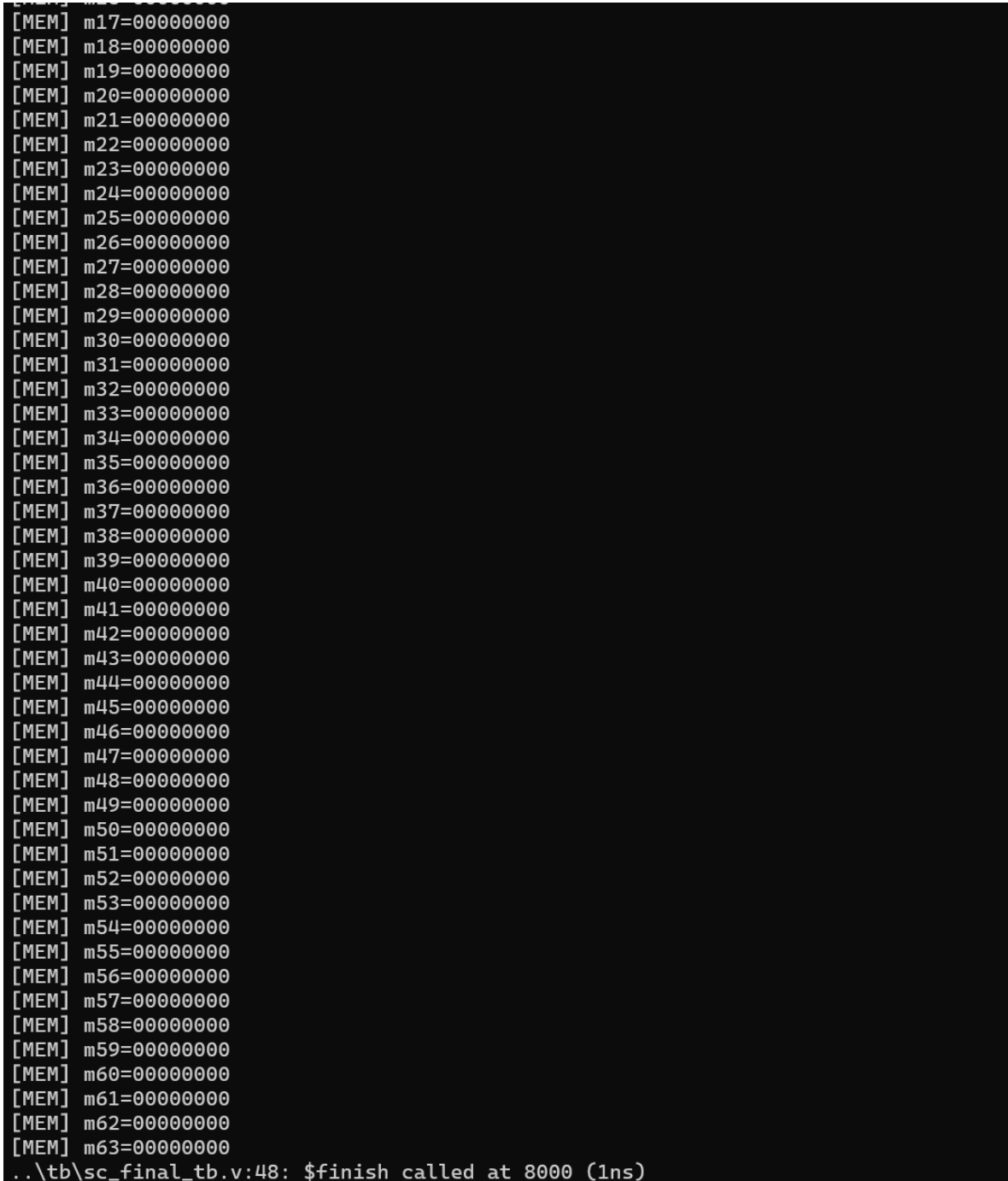
图 5 测试 2: 流水线 CPU 跑 Test_30_Instr.dat 的内存截图

```
[MEM] m11=00000000
[MEM] m12=00000000
[MEM] m13=00000000
[MEM] m14=00000000
[MEM] m15=00000000
[MEM] m16=00000000
[MEM] m17=00000000
[MEM] m18=00000000
[MEM] m19=00000000
[MEM] m20=00000000
[MEM] m21=00000000
[MEM] m22=00000000
[MEM] m23=00000000
[MEM] m24=00000000
[MEM] m25=00000000
[MEM] m26=00000000
[MEM] m27=00000000
[MEM] m28=00000000
[MEM] m29=00000000
[MEM] m30=00000000
[MEM] m31=00000000
[MEM] m32=00000000
[MEM] m33=00000000
[MEM] m34=00000000
[MEM] m35=00000000
[MEM] m36=00000000
[MEM] m37=00000000
[MEM] m38=00000000
[MEM] m39=00000000
[MEM] m40=00000000
[MEM] m41=00000000
[MEM] m42=00000000
[MEM] m43=00000000
[MEM] m44=00000000
[MEM] m45=00000000
[MEM] m46=00000000
[MEM] m47=00000000
[MEM] m48=00000000
[MEM] m49=00000000
[MEM] m50=00000000
[MEM] m51=00000000
[MEM] m52=00000000
[MEM] m53=00000000
[MEM] m54=00000000
[MEM] m55=00000000
[MEM] m56=00000000
[MEM] m57=00000000
[MEM] m58=00000000
[MEM] m59=00000000
[MEM] m60=00000000
[MEM] m61=00000000
[MEM] m62=00000000
[MEM] m63=00000000
..\tb\pl_final_tb.v:45: $finish called at 725000 (1ps)
```

图 6 测试 3: 单周期 CPU 跑学号排序程序的寄存器截图, x15 为排序结果

```
.
[SNAPSHOT] registers
[REG] x0=00000000
[REG] x1=ffff0004
[REG] x2=ffff000c
[REG] x3=00000008
[REG] x4=00000000
[REG] x5=00000100
[REG] x6=00ffffff
[REG] x7=00000000
[REG] x8=03000000
[REG] x9=0000001c
[REG] x10=0000001c
[REG] x11=00000008
[REG] x12=00000007
[REG] x13=00000000
[REG] x14=00000000
[REG] x15=03345578
[REG] x16=00000000
[REG] x17=00000000
[REG] x18=00000000
[REG] x19=00000000
[REG] x20=00000000
[REG] x21=00000000
[REG] x22=00000000
[REG] x23=00000000
[REG] x24=00000000
[REG] x25=00000000
[REG] x26=00000000
[REG] x27=00000000
[REG] x28=00000000
[REG] x29=00000000
[REG] x30=00000000
[REG] x31=00000000
[SNAPSHOT] memory
[MEM] m0=00000000
[MEM] m1=00000000
[MEM] m2=00000000
[MEM] m3=00000000
[MEM] m4=00000000
[MEM] m5=00000000
[MEM] m6=00000000
[MEM] m7=00000000
[MEM] m8=00000000
[MEM] m9=00000000
[MEM] m10=00000000
[MEM] m11=00000000
[MEM] m12=00000000
[MEM] m13=00000000
[MEM] m14=00000000
```

图 7 测试 3: 单周期 CPU 跑学号排序程序的内存窗口截图



```
[MEM] m17=00000000
[MEM] m18=00000000
[MEM] m19=00000000
[MEM] m20=00000000
[MEM] m21=00000000
[MEM] m22=00000000
[MEM] m23=00000000
[MEM] m24=00000000
[MEM] m25=00000000
[MEM] m26=00000000
[MEM] m27=00000000
[MEM] m28=00000000
[MEM] m29=00000000
[MEM] m30=00000000
[MEM] m31=00000000
[MEM] m32=00000000
[MEM] m33=00000000
[MEM] m34=00000000
[MEM] m35=00000000
[MEM] m36=00000000
[MEM] m37=00000000
[MEM] m38=00000000
[MEM] m39=00000000
[MEM] m40=00000000
[MEM] m41=00000000
[MEM] m42=00000000
[MEM] m43=00000000
[MEM] m44=00000000
[MEM] m45=00000000
[MEM] m46=00000000
[MEM] m47=00000000
[MEM] m48=00000000
[MEM] m49=00000000
[MEM] m50=00000000
[MEM] m51=00000000
[MEM] m52=00000000
[MEM] m53=00000000
[MEM] m54=00000000
[MEM] m55=00000000
[MEM] m56=00000000
[MEM] m57=00000000
[MEM] m58=00000000
[MEM] m59=00000000
[MEM] m60=00000000
[MEM] m61=00000000
[MEM] m62=00000000
[MEM] m63=00000000
..\tb\sc_final_tb.v:48: $finish called at 8000 (1ns)
```

图 8 测试 4: 流水线 CPU 跑学号排序程序的写内存与寄存器截图

```
WARNING: ..\tb\pl_final_tb.v:22: $readmemh(E:\computerlab\ComputerDesign\
memaddr = 00000180, memdata = 54873530
memaddr = 00000184, memdata = 03345578
memaddr = 00000184, memdata = 03345578
memaddr = 00000184, memdata = 03345578
memaddr = 00000184, memdata = 03345578
memaddr = 00000184, memdata = 03345578
memaddr = 00000184, memdata = 03345578
memaddr = 00000184, memdata = 03345578
memaddr = 00000184, memdata = 03345578
memaddr = 00000184, memdata = 03345578
memaddr = 00000184, memdata = 03345578
memaddr = 00000184, memdata = 03345578
memaddr = 00000184, memdata = 03345578
memaddr = 00000184, memdata = 03345578
memaddr = 00000184, memdata = 03345578
memaddr = 00000184, memdata = 03345578
memaddr = 00000184, memdata = 03345578
memaddr = 00000184, memdata = 03345578
memaddr = 00000184, memdata = 03345578
memaddr = 00000184, memdata = 03345578
memaddr = 00000184, memdata = 03345578
memaddr = 00000184, memdata = 03345578
memaddr = 00000184, memdata = 03345578
memaddr = 00000184, memdata = 03345578
memaddr = 00000184, memdata = 03345578
memaddr = 00000184, memdata = 03345578
memaddr = 00000184, memdata = 03345578
memaddr = 00000184, memdata = 03345578
memaddr = 00000184, memdata = 03345578
[SNAPSHOT] registers
[REG] x0=00000000
[REG] x1=ffff0004
[REG] x2=ffff000c
[REG] x3=00000008
[REG] x4=00000000
[REG] x5=00000100
[REG] x6=00ffffff
[REG] x7=00000000
[REG] x8=03000000
[REG] x9=0000001c
[REG] x10=0000001c
[REG] x11=00000008
[REG] x12=00000007
[REG] x13=00000000
[REG] x14=00000000
[REG] x15=03345578
[REG] x16=00000000
[REG] x17=00000000
```

图 9 测试 4：流水线 CPU 跑学号排序程序的内存窗口截图（上半部分）

```
[REG] x18=00000000
[REG] x19=00000000
[REG] x20=00000000
[REG] x21=00000000
[REG] x22=00000000
[REG] x23=00000000
[REG] x24=00000000
[REG] x25=00000000
[REG] x26=00000000
[REG] x27=00000000
[REG] x28=00000000
[REG] x29=00000000
[REG] x30=00000000
[REG] x31=00000000
[SNAPSHOT] memory
[MEM] m0=00000000
[MEM] m1=00000000
[MEM] m2=00000000
[MEM] m3=00000000
[MEM] m4=00000000
[MEM] m5=00000000
[MEM] m6=00000000
[MEM] m7=00000000
[MEM] m8=00000000
[MEM] m9=00000000
[MEM] m10=00000000
[MEM] m11=00000000
[MEM] m12=00000000
[MEM] m13=00000000
[MEM] m14=00000000
[MEM] m15=00000000
[MEM] m16=00000000
[MEM] m17=00000000
[MEM] m18=00000000
[MEM] m19=00000000
[MEM] m20=00000000
[MEM] m21=00000000
[MEM] m22=00000000
[MEM] m23=00000000
[MEM] m24=00000000
[MEM] m25=00000000
[MEM] m26=00000000
[MEM] m27=00000000
[MEM] m28=00000000
[MEM] m29=00000000
[MEM] m30=00000000
[MEM] m31=00000000
[MEM] m32=00000000
[MEM] m33=00000000
[MEM] m34=00000000
[MEM] m35=00000000
[MEM] m36=00000000
[MEM] m37=00000000
```

图 10 测试 4：流水线 CPU 跑学号排序程序的内存窗口截图（下半部分）

```

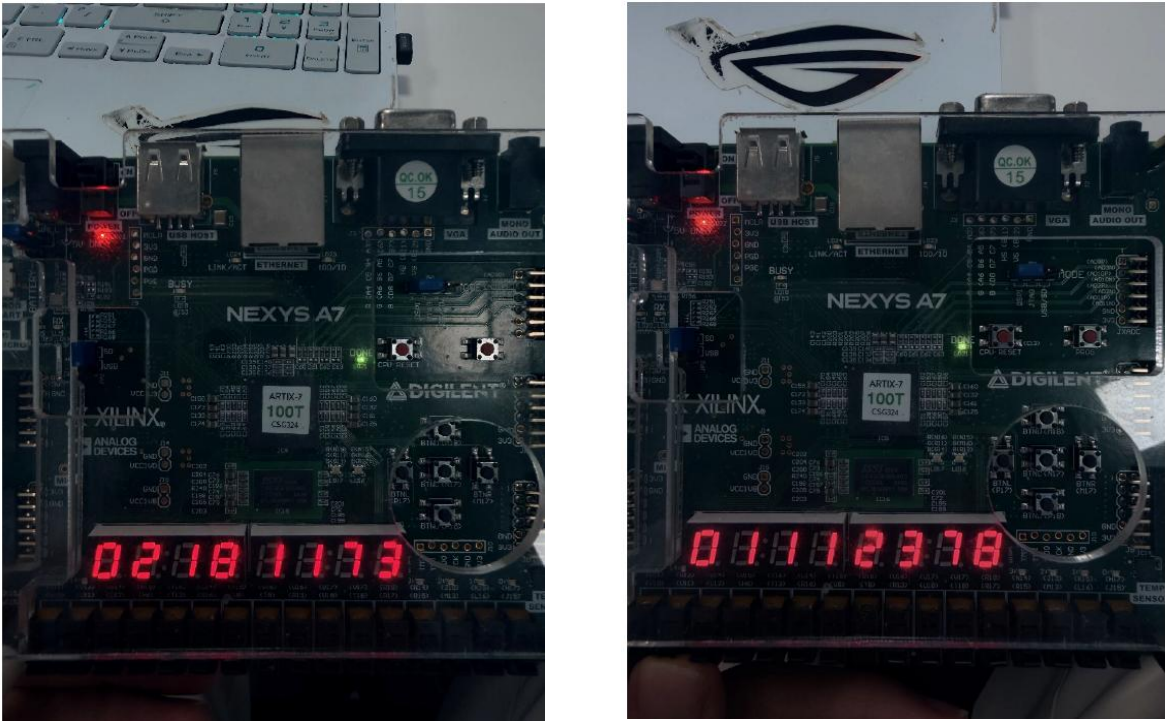
[MEM] m29=00000000
[MEM] m30=00000000
[MEM] m31=00000000
[MEM] m32=00000000
[MEM] m33=00000000
[MEM] m34=00000000
[MEM] m35=00000000
[MEM] m36=00000000
[MEM] m37=00000000
[MEM] m38=00000000
[MEM] m39=00000000
[MEM] m40=00000000
[MEM] m41=00000000
[MEM] m42=00000000
[MEM] m43=00000000
[MEM] m44=00000000
[MEM] m45=00000000
[MEM] m46=00000000
[MEM] m47=00000000
[MEM] m48=00000000
[MEM] m49=00000000
[MEM] m50=00000000
[MEM] m51=00000000
[MEM] m52=00000000
[MEM] m53=00000000
[MEM] m54=00000000
[MEM] m55=00000000
[MEM] m56=00000000
[MEM] m57=00000000
[MEM] m58=00000000
[MEM] m59=00000000
[MEM] m60=00000000
[MEM] m61=00000000
[MEM] m62=00000000
[MEM] m63=00000000
.\tb\pl_final_tb.v:45: $finish called at 9045000 (1ps)

```


七、开发板运行结果

开发板照片用于证明程序可以在 Nexys A7/Nexys4 DDR 板上完成七段数码管显示。照片中可见数码管显示原始学号与排序/演示结果，说明显示通路、时钟复位和板级约束能够正常工作。

图 11 开发板照片：原始学号显示与排序/演示结果显示



八、代码修改与 diff 说明

本次提交的关键修改集中在单周期和流水线 CPU 的译码、ALU、立即数扩展、下一条 PC 计算、流水寄存器传递和冒险处理。完整源码后续放入 GitHub 公开仓库，报告只展示主要 diff 截图和修改摘要。

文件/模块	主要修改	用途
single-cycle/NPC.v	新增 RS1 输入和 NPC_JALR 目标计算	支持 jalr 使用 rs1+imm 跳转并清除最低位。
single-cycle/SCCPU.v	NPC 实例连接 .RS1(RD1)	把 RF 读出的 rs1 数据送给 NPC。
single-cycle/alu.v	扩展 bne/blt/bge/bltu/bgeu、slt/sltu、移位等 ALUOp	完成运算、比较和分支条件判断。

single-cycle/ctrl.v	改为 always @(*) + case 译码	更清晰地按 opcode/funct3/funct7 产生控制信号，便于扩展指令。
pipeline/EXT.v / NPC.v	增加 B/J 立即数和 JumpPC/RS1	支持流水线分支、jal、jalr 目标地址。
pipeline/PLCPU.v	增加数据转发、jump_flush、流水寄存器中文注释	解决 RAW 数据相关和控制冒险，便于答辩解释。

图 12 diff 截图：single-cycle/NPC.v 增加 RS1 和 NPC_JALR

```
modified single-cycle/NPC.v +3 -1
1 1 `include "ctrl_encode_def.v"
2 2
3 - module NPC( PC, NPCOp, IMM, NPC ); // next pc module
3 + module NPC( PC, NPCOp, IMM, RS1, NPC ); // next pc module
4 4 input [31:0] PC; // pc
5 5 input [2:0] NPCOp; // next pc operation
6 6 input [31:0] IMM; // immediate
7 + input [31:0] RS1; // rs1 data for jalr
7 8 output reg [31:0] NPC; // next pc
8 9
9 10 wire [31:0] PCPLUS4;
...
14 15 `NPC_PLUS4: NPC = PCPLUS4; // NPC computes addr
15 16 `NPC_BRANCH: NPC = PC+IMM; //B type, NPC computes addr
16 17 `NPC_JUMP: NPC = PC+IMM; //J type, NPC computes addr
18 + `NPC_JALR: NPC = (RS1 + IMM) & 32'hffff_fffe;
17 19 default: NPC = PCPLUS4;
18 20 endcase
19 21 end
...
```

图 13 diff 截图：PC/RF 删除调试 \$write 输出

```
modified single-cycle/PC.v +0 -1
...
9 PC <= 32'h0000_0000;
10 else
11 PC <= NPC;
12 - $write("\n pc = %h: ", PC);
13 end
14 endmodule
...

modified single-cycle/RF.v +0 -1
...
21 if (RFwr) begin
22 if (A3 != 5'b0) begin
23 rf[A3] <= WD;
24 - $write("\n %d = %h ", A3, WD);
25 end
26 end
27 end
...
```

图 14 diff 截图: SCCPU 中 NPC 连接 RS1(RD1)

modified single-cycle/SCCPU.v		+2 -2	
71		71	...
72		72	);
73		73	// instantiation of pc unit
			PC U_PC(.clk(clk), .rst(reset), .NPC(NPC),
			.PC(PC_out));
74		-	NPC U_NPC(.PC(PC_out), .NPCOp(NPCOp),
			.IMM(immout), .NPC(NPC));
		74	+ NPC U_NPC(.PC(PC_out), .NPCOp(NPCOp),
			.IMM(immout), .RS1(RD1), .NPC(NPC));
75		75	EXT U_EXT{
76		76	.imm(imm), .simm(simm), .bimm(bimm),
			.uimm(uimm), .jimm(jimm),
77		77	.EXTOp(EXTOp), .immout(immout)
			...;
99		99	endcase
100		100	end
101		101	
102		-	endmodule
		102	+ endmodule

GitHub 公开仓库: <https://github.com/yuxuanhuang641-crypto/computer-design-practice-final-2024302181173>

九、问题回答汇总

9.1 如果老师问: op、funct3、funct7 是什么?

它们是 RISC-V 指令编码中的字段。CPU 从 im 取出 32 位 instr 后, 用 instr[6:0] 得到 opcode, 用 instr[14:12] 得到 funct3, 用 instr[31:25] 得到 funct7。opcode 先判断大类, 比如 R-type、I-type、branch、jal、jalr; funct3/funct7 再区分同一大类里的具体指令, 例如 add/sub 都是 R-type 且 funct3=000, 但 funct7 不同。

9.2 如果老师问: 为什么 ctrl 要改成 always + case?

原来的写法是很多 wire 条件和 assign 拼起来, 适合少量指令; 扩展到十几条指令后会很难维护, 也容易漏控制信号。改成 always @(*) 先给默认值, 再按 opcode case 分支, 每类指令内部再看 funct3/funct7, 结构更像真实译码器, 也能保证未识别指令默认不写寄存器、不写内存。

9.3 如果老师问: dm 是做什么的?

dm 就是数据内存, 只在 lw/sw 这类访存指令中真正起作用。sw 时 CPU 给地址和写数据, MemWrite=1 后在时钟沿写入; lw 时 CPU 给地址, dm 输出读出的数据, 后面由 WDSel=FromMEM 写回寄存器。它和 im 的区别是 im 存指令、通常只读; dm 存程序运行的数据, 既能读也能写。

9.4 如果老师问: WB_WDSel 是什么?

WB_WDSel 是流水线 WB 阶段的写回数据选择信号。它决定寄存器堆 WD 来自哪里：普通运算指令选择 ALU 结果，lw 选择内存数据，jal/jalr 选择 PC+4。因为流水线里控制信号要跟着指令走，所以 WDSel 在 ID 阶段产生后要一路保存到 MEM/WB，最终在 WB 阶段使用。

9.5 如果老师问：Test_30 和学号排序分别测什么？

Test_30_Instr.dat 主要验证新增指令点是否正确，包括算术逻辑、比较、移位、load/store、分支和 jal/jalr。学号排序程序更像完整程序，会反复使用循环、分支、子过程调用、jalr 返回、连续数据相关和内存读写，因此更能暴露流水线冒险处理是否可靠。

十、总结

本次最终验收完成了单周期 CPU 与五级流水线 CPU 的扩展指令实现。单周期部分重点是译码、ALU 和 NPC 的组合逻辑补全；流水线部分除了复用这些功能，还额外处理了控制信号跨阶段保存、RAW 数据转发和跳转 flush。四组老师指定测试和自动评分均达到预期，上板照片也完成了数码管显示验证。

答辩时如果老师要求现场指出代码位置，可以优先打开：ctrl.v 看 opcode/funct3/funct7 译码，alu.v 看 add/slt/bne 的运算，NPC.v 看 jalr 的 PC 目标，PLCPU.v 看 forwarding、IF/ID、ID/EX、EX/MEM、MEM/WB 和 jump_flush。